Module 03

Why DBMS?/2

Milav Dabgar

www.milav.in

March 17, 2026

Module Recap

- Evolution of Data and Records Management
- History of DBMS

Module Objectives

- Comparison of File based data management and DBMS

Module Outline

- File handling by Python viz-a-viz DBMS - Bank Transaction example

Module Outline

- File handling by Python viz-a-viz DBMS - Bank Transaction example
- Parameterized Comparison

File Systems vs Databases

Case study: A bank transaction

Banking Transaction System

Consider a simple banking system where a person can open a new account, transfer fund to an existing account and check the history of all her transactions till date.

Case study: A bank transaction

Banking Transaction System

Consider a simple banking system where a person can open a new account, transfer fund to an existing account and check the history of all her transactions till date.

The application performs the following checks:

- If the account balance is not enough, it will not allow the fund transfer
- If the account numbers are not correct, it will flash a message and terminate the transaction.

Case study: A bank transaction

Banking Transaction System

Consider a simple banking system where a person can open a new account, transfer fund to an existing account and check the history of all her transactions till date.

The application performs the following checks:

- If the account balance is not enough, it will not allow the fund transfer
- If the account numbers are not correct, it will flash a message and terminate the transaction.
- If a transaction is successful, it prints a confirmation message.

Case study: A bank transaction (2)

We will use this banking transaction system to compare various features of a file-based (spreadsheet/.csv files) implementation viz-a-viz a DBMS-based implementation

- Account details are stored in

In the following slides we discuss a fund transfer transaction.

Source:

`https://github.com/bhaskariitm/transition-from-files-to-db/tree/main`

Case study: A bank transaction (2)

We will use this banking transaction system to compare various features of a file-based (spreadsheet/.csv files) implementation viz-a-viz a DBMS-based implementation

- Account details are stored in
  - `Accounts.csv` for file-based implementation

In the following slides we discuss a fund transfer transaction.

Source:

`https://github.com/bhaskariitm/transition-from-files-to-db/tree/main`

Case study: A bank transaction (2)

We will use this banking transaction system to compare various features of a file-based (spreadsheet/.csv files) implementation viz-a-viz a DBMS-based implementation

- Account details are stored in
  - `Accounts.csv` for file-based implementation
  - `Accounts` table for DBMS implementation

In the following slides we discuss a fund transfer transaction.

Source:

`https://github.com/bhaskariitm/transition-from-files-to-db/tree/main`

Case study: A bank transaction (2)

We will use this banking transaction system to compare various features of a file-based (spreadsheet/.csv files) implementation viz-a-viz a DBMS-based implementation

- Account details are stored in
  - `Accounts.csv` for file-based implementation
  - `Accounts` table for DBMS implementation
- The transaction details are stored in

In the following slides we discuss a fund transfer transaction.

Source:

`https://github.com/bhaskariitm/transition-from-files-to-db/tree/main`

Case study: A bank transaction (2)

We will use this banking transaction system to compare various features of a file-based (spreadsheet/.csv files) implementation viz-a-viz a DBMS-based implementation

- Account details are stored in
  - `Accounts.csv` for file-based implementation
  - `Accounts` table for DBMS implementation
- The transaction details are stored in
  - `Ledger.csv` file for file-based implementation

In the following slides we discuss a fund transfer transaction.

Source:

`https://github.com/bhaskariitm/transition-from-files-to-db/tree/main`

Case study: A bank transaction (2)

We will use this banking transaction system to compare various features of a file-based (spreadsheet/.csv files) implementation viz-a-viz a DBMS-based implementation

- Account details are stored in
  - `Accounts.csv` for file-based implementation
  - `Accounts` table for DBMS implementation
- The transaction details are stored in
  - `Ledger.csv` file for file-based implementation
  - `Ledger` table for DBMS implementation

In the following slides we discuss a fund transfer transaction.

Source:

`https://github.com/bhaskariitm/transition-from-files-to-db/tree/main`

Python viz-a-viz SQL

Bank Transaction: Python viz-a-viz SQL

Python

```
def begin_Transaction(creditAcc, debitAcc, amount):
    temp = []
    success = 0
    # Open file handles to retrieve and store transaction data
    f_obj_Account1 = open('Accounts.csv', 'r')
    f_reader1 = csv.DictReader(f_obj_Account1)

    f_obj_Account2 = open('Accounts.csv', 'r')
    f_reader2 = csv.DictReader(f_obj_Account2)

    f_obj_Ledger = open('Ledger.csv', 'a+')
    f_writer = csv.DictWriter(f_obj_Ledger, fieldnames=col_name_Ledger)
```

SQL

// Handled implicitly by the DBMS

Bank Transaction: Python viz-a-viz SQL (2)

Python

```
try:
    for sRec in f_reader1:
        #CONDITION CHECK FOR ENOUGH BALANCE
        if sRec["AcctNo"] == debitAcc and int(sRec["Balance"]) > int(amt):
            ...
```

SQL

```
do $$
begin
    amt = 5000;
    sendVal = '1800090';
    recVal = '1800100';

    select balance from accounts
        into sbalance
        where account_no = sendVal;

    if sbalance < amt then
        ...
$$
```

Bank Transaction: Python viz-a-viz SQL (3)

Python

```
# in continuation...
        for rRec in f_reader2:
            if rRec["AcctNo"] == creditAcc:
                sRec["Balance"] = str(int(sRec["Balance"]) - int(amt)) # DEBIT
                temp.append(sRec)
                ...
```

SQL

```
    # in continuation...
    if sbalance < amt then
        raise notice "Insufficient bala
        update accounts
            set balance = balance - amt
            where account_no = sendVal;
        ...
$$
```

Bank Transaction: Python viz-a-viz SQL (4)

Python

```
# in continuation...
                # Critical point
                f_writer.writerow({
                    "Acct1":sRec["AcctNo"],
                    "Acct2":rRec["AcctNo"],
                    "Amount":amt, "D/C":"D"})

                rRec["Balance"] = str(int(rRec["Balance"]) + int(amt)) # CREDI
                temp.append(rRec)
                ...
```

SQL

```
# in continuation...
        insert into ledger(sendAc, recA
            values(sendVal, recVal, amt

        update accounts
            set balance = balance + amt
            where account_no = recVal;
        $$
```

Bank Transaction: Python viz-a-viz SQL (5)

Python

```
# in continuation...
                f_writer.writerow({
                    "Acct1":rRec["AcctNo"],
                    "Acct2":sRec["AcctNo"],
                    "Amount":amt,"D/C":"C"})
                success = success + 1
                break

        f_obj_Account1.seek(0)
        next(f_obj_Account1)
        for record in f_reader1:
            if record["AcctNo"] != temp[0]["AcctNo"] and record["AcctNo"] != t
except:
    print("Wrong input entered !!!")
```

SQL

```
# in continuation...
            insert into ledger(sendAc, recA
                values(recVal, sendVal, amt
            commit;
            raise notice "Successful";
        end if;
end;
$$
```

Bank Transaction: Python viz-a-viz SQL (6)

Python

```
#Writing back to the file
f_obj_Account1.close()
f_obj_Account2.close()
f_obj_Ledger.close()

if success == 1:
    f_obj_Account = open('Accounts.csv', 'w+', newline='')
    f_writer = csv.DictWriter(f_obj_Account, fieldnames=col_name_Account)
    f_writer.writeheader()
    for data in temp:
        f_writer.writerow(data)
    f_obj_Account.close()
    print("Transaction is successful !!")
else:
```

SQL

// Handled implicitly by the DBMS

Parameterized Comparison

Comparison

Parameter	File Handling via Python	DBMS
Scalability (data amount)	Very difficult to handle insert, update and querying of records	In-built features to provide high scalability for a large number of records
Scalability (structure change)	Extremely difficult to change the structure of records as in the case of adding or removing attributes	Adding or removing attributes can be done seamlessly using simple SQL queries
Time of execution	In seconds	In milliseconds
Persistence	Data processed using temporary data structures have to be manually updated to the file	Data persistence is ensured via automatic, system induced mechanisms
Robustness	Ensuring robustness of data has to be done manually	Backup, recovery and restore need minimum manual intervention
Security	Difficult to implement in Python (Security at OS level)	User-specific access at database level
Programmer's productivity	Most file access operations involve extensive coding to ensure persistence, robustness and security of data	Standard and simple built-in queries reduce the effort involved in coding thereby increasing a programmer's

Scalability

File handling via Python

- **Number of records:** As the # of records increases, efficiency reduces (OS limitations).

Scalability

File handling via Python

- **Number of records:** As the # of records increases, efficiency reduces (OS limitations).
- **Structural Change:** Initializing new attributes or deleting existing ones is manual and error-prone.

Scalability

File handling via Python

- **Number of records:** As the # of records increases, efficiency reduces (OS limitations).
- **Structural Change:** Initializing new attributes or deleting existing ones is manual and error-prone.

DBMS

- **Number of records:** Built to scale efficiently via indexing.

Scalability

File handling via Python

- **Number of records:** As the # of records increases, efficiency reduces (OS limitations).
- **Structural Change:** Initializing new attributes or deleting existing ones is manual and error-prone.

DBMS

- **Number of records:** Built to scale efficiently via indexing.
- **Structural Change:** Seamlessly add default values or use constraints to handle safe removal.

Time and Efficiency

File handling via Python

- Low effort to implement file handlers

Time and Efficiency

File handling via Python

- Low effort to implement file handlers
- Processing a 1GB file might take **seconds**

Time and Efficiency

File handling via Python

- Low effort to implement file handlers
- Processing a 1GB file might take **seconds**

DBMS

- Expensive & time consuming to configure

Time and Efficiency

File handling via Python

- Low effort to implement file handlers
- Processing a 1GB file might take **seconds**

DBMS

- Expensive & time consuming to configure
- Processing a 1GB file takes **milliseconds**

Time and Efficiency

File handling via Python

- Low effort to implement file handlers
- Processing a 1GB file might take **seconds**

DBMS

- Expensive & time consuming to configure
- Processing a 1GB file takes **milliseconds**
- High volume $\rightarrow$ Initialization time becomes negligible vs query time saved.

Persistence, Robustness, Security

File handling via Python

- **Persistence:** Manual writes to disk

Persistence, Robustness, Security

File handling via Python

- **Persistence:** Manual writes to disk
- **Robustness:** Crashes lead to inconsistency

Persistence, Robustness, Security

File handling via Python

- **Persistence:** Manual writes to disk
- **Robustness:** Crashes lead to inconsistency
- **Security:** OS level authentication only

Persistence, Robustness, Security

File handling via Python

- **Persistence:** Manual writes to disk
- **Robustness:** Crashes lead to inconsistency
- **Security:** OS level authentication only

Persistence, Robustness, Security

File handling via Python

- **Persistence:** Manual writes to disk
- **Robustness:** Crashes lead to inconsistency
- **Security:** OS level authentication only

DBMS

- **Persistence:** Automatic system logs
- **Robustness:** Built-in rollback and recovery

Persistence, Robustness, Security

File handling via Python

- **Persistence:** Manual writes to disk
- **Robustness:** Crashes lead to inconsistency
- **Security:** OS level authentication only

DBMS

- **Persistence:** Automatic system logs
- **Robustness:** Built-in rollback and recovery
- **Security:** Granular, user-specific access

Programmer's Productivity

File handling via Python

- Constraints enforced manually

Programmer's Productivity

File handling via Python

- Constraints enforced manually
- High effort validating logic during inserts/updates

Programmer's Productivity

File handling via Python

- Constraints enforced manually
- High effort validating logic during inserts/updates
- Massive effort scaling for big data

Programmer's Productivity

File handling via Python

- Constraints enforced manually
- High effort validating logic during inserts/updates
- Massive effort scaling for big data

Programmer's Productivity

File handling via Python

- Constraints enforced manually
- High effort validating logic during inserts/updates
- Massive effort scaling for big data

DBMS

- Configured by specialised DBA
- In-built constraints handle sanity inherently

Programmer's Productivity

File handling via Python

- Constraints enforced manually
- High effort validating logic during inserts/updates
- Massive effort scaling for big data

DBMS

- Configured by specialised DBA
- In-built constraints handle sanity inherently
- Seamless handling of terabytes of data

Cost Vs Complexity

File handling via Python

- **Pros:** Extensive math support in Python

Cost Vs Complexity

File handling via Python

- **Pros:** Extensive math support in Python
- **Cost:** Cheap setup without dedicated specialists

Cost Vs Complexity

File handling via Python

- **Pros:** Extensive math support in Python
- **Cost:** Cheap setup without dedicated specialists

Cost Vs Complexity

File handling via Python

- **Pros:** Extensive math support in Python
- **Cost:** Cheap setup without dedicated specialists

DBMS

- **Cons:** SQL has limited advanced arithmetic logic
- **Cost:** Expensive servers, licenses, and DBAs required

Module Summary

- Elucidated the difference between File handling by Python viz-a-viz DBMS through an Bank Transaction example
- Parameterized Comparison

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.